ACADEMIC CENTRAL

PRACTICE EXAMINATION

NC(V) LEVEL 3

INTRODUCTION TO ROBOTICS

PAPER 2 · PRACTICE D · MARKING GUIDELINE

MARKS: 80

Marking guideline for the Academic Central NC(V) Level 3 Introduction to Robotics paper 2 · practice d.

Written by Academic Central for practice, following the NC(V) subject guidelines for NC(V) Level 3. It is not a Department of Higher Education and Training marking guideline.

How to mark: each ✓ is one mark. Practical work is not marked strictly on syntax. Give marks for correct logic, correct pins and connections, sensible comments and neat work. Accept other pin choices, layouts and program structures that work.

QUESTION 1

1.1 IR sensor: VCC to 5 V ✓, GND to GND ✓, OUT to a digital pin ✓. HC-SR04: VCC and GND ✓, Trig to a digital pin ✓, Echo to a digital pin ✓. Servo: signal to a PWM pin ✓, red to 5 V and brown to GND ✓. LED through the 220 Ω resistor to a digital pin ✓✓, cathode to GND ✓. Buzzer + to a digital pin and − to GND ✓. 5 V and GND rails from the Arduino ✓; neat, no short circuits ✓. (−1 for each wrong connection, −2 for any short circuit) (14)

1.2 Correct symbols ✓, e.g.: (14)

```
START ✓
  ↓
Set up pins and servo, close lid ✓
  ↓
Measure distance with the HC-SR04, work out fill % ◄──────────────┐ ✓ ✓
  ↓                                                               │
◇ Distance 0 or more than 100 cm? — Yes → "Sensor error", lid closed ✓ ✓
  ↓ No                                                            │
◇ Distance less than 8 cm (full)? — Yes → LED on, beep 3 times, lid stays closed ✓ ✓
  ↓ No                                                            │
LED off ✓                                                         │
  ↓                                                               │
◇ Hand detected (IR OUT LOW)? — Yes → open lid 90°, wait 3 s, close lid ✓ ✓
  ↓ No                                                            │
  └──────────────────────────────────────── loop back ✓ ──────────┘
```

1.3 Circuit saved and printed / presented ✓; flowchart saved and printed / presented ✓ (2)

[30]

QUESTION 2

2.1 Servo library and object ✓; pin modes for Trig, Echo, IR, LED and buzzer ✓; Serial.begin ✓ and lid closed at the start ✓; trigger pulse (LOW, HIGH 10 µs, LOW) ✓; pulseIn on Echo ✓; distance calculation ✓; fill % calculated ✓ and printed ✓; error check with "Sensor error" ✓; full condition ✓ with LED on ✓ and three beeps in a loop ✓; lid does not open when full ✓; LED off otherwise ✓; hand check (LOW) ✓; open 90°, wait 3 s, close ✓; comments ✓, e.g. (18)

```
#include <Servo.h>
Servo lid;
const int irPin = 2, trigPin = 7, echoPin = 8, ledPin = 12, buzzerPin = 13;
const int binDepth = 40;                 // cm

void setup() {
  pinMode(irPin, INPUT);
  pinMode(trigPin, OUTPUT);
```

```
  pinMode(echoPin, INPUT);
  pinMode(ledPin, OUTPUT);
  pinMode(buzzerPin, OUTPUT);
  lid.attach(9);
  lid.write(0);                          // start with the lid closed
  Serial.begin(9600);
}

void loop() {
  digitalWrite(trigPin, LOW);  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH); delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  long duration = pulseIn(echoPin, HIGH);
  int distance = duration * 0.034 / 2;   // cm

  if (distance == 0 || distance > 100) { // impossible reading
    Serial.println("Sensor error");
    lid.write(0);
  } else {
    int fill = (binDepth - distance) * 100 / binDepth;
    Serial.print("Fill level: "); Serial.print(fill); Serial.println(" %");
    if (distance < 8) {                  // bin is full
      digitalWrite(ledPin, HIGH);
      for (int i = 0; i < 3; i++) {      // beep three times
        digitalWrite(buzzerPin, HIGH); delay(200);
        digitalWrite(buzzerPin, LOW);  delay(200);
      }
    } else {
      digitalWrite(ledPin, LOW);
      if (digitalRead(irPin) == LOW) {   // hand in front of the bin
        lid.write(90);                   // open
        delay(3000);
        lid.write(0);                    // close
      }
    }
  }
  delay(200);
}
```

2.2 Program saved ✓ and printed / presented ✓ (2)

[20]

QUESTION 3

3.1 Three new tests on c ✓✓✓ ('1', '2' and '3') that set speed to 100 ✓, 170 ✓ and 255 ✓; the speed variable is used by drive() so the next movement uses it ✓; the speed commands do not move the car ✓; correct syntax / logic ✓ (9)

3.2 unsigned long variable for the time of the last command ✓✓; updated with millis() every time a character is received ✓✓; in loop(): if millis() − lastCommand > 2000 ✓✓ the car stops ✓ by calling drive(LOW, LOW, LOW, LOW) ✓; the stop does not block later commands ✓ (9)

3.3 const int horn = 4; ✓ pinMode(horn, OUTPUT); ✓ test for 'H' ✓; buzzer HIGH ✓, delay(500) ✓, buzzer LOW ✓; inside the Serial.available() block ✓; horn does not change the movement ✓; correct logic ✓ (9)

```
const int horn = 4;                       // 3.3
unsigned long lastCommand = 0;            // 3.2

// in setup():  pinMode(horn, OUTPUT);

void loop() {
  if (Serial.available() > 0) {
    char c = Serial.read();
    lastCommand = millis();               // 3.2: a command was received
    if (c == 'F') drive(HIGH, LOW, HIGH, LOW);
    if (c == 'B') drive(LOW, HIGH, LOW, HIGH);
    if (c == 'L') drive(LOW, HIGH, HIGH, LOW);
    if (c == 'R') drive(HIGH, LOW, LOW, HIGH);
    if (c == 'S') drive(LOW, LOW, LOW, LOW);
    if (c == '1') speed = 100;            // 3.1
    if (c == '2') speed = 170;
    if (c == '3') speed = 255;
    if (c == 'H') {                       // 3.3: horn
      digitalWrite(horn, HIGH);
      delay(500);
      digitalWrite(horn, LOW);
    }
  }
  if (millis() - lastCommand > 2000) {    // 3.2: no command for 2 s
    drive(LOW, LOW, LOW, LOW);            // stop for safety
  }
}
```

3.4 Final program saved ✓, printed / presented ✓, with comments showing the changes ✓ (3)

[30]

TOTAL: 80

END OF MARKING GUIDELINE